```
[BindProperty(Name="e-mail")]
public string Email { get; set; }
```

If you have a large number of `PageModel` properties you want to include in model binding, you can decorate the `PageModel` class with a `BindProperties` (plural) attribute:

```
[BindProperties]
public class IndexModel : PageModel
```

This approach must be used with care. It will result in *all* public properties in the `PageModel` class being included in model binding, which could expose your application to an overposting attack.

Overposting attacks

Overposting, also referred to as mass assignment, is a vulnerability that happens when the user is able to modify items of data they should not have access to. Imagine you are creating a checkout page where the user confirms details of their purchase. Clearly you don't want them to be able to modify the price of the item, so there is no form field for this piece of data. However, let's assume you have included the `Price` as a public property on the `PageModel`, where the `PageModel` class is decorated with the `BindProperties` attribute, thereby opting in all public properties, including the `Price` property, as binding targets.

It is perfectly possible for a web-savvy user to craft their own HTTP request using basic development tools, such as Postman or even the browser developer tools (which we will look at in chapter 11), and include a form item for the price. Because the `Price` property is exposed to the model binder, the user is able to modify the value. Before you know it, those $300 headphones on your site are being sold for $3.

5.2.3 Binding complex objects